Café Inventory Management System

The purpose of this program is to design and implement a program that manages the products of a Café. This program is able to read products from a text file, add products up to a limit, display the items, calculate the inventory value of each product, search a specific product, display the low-stock products, and save the information back to the file.

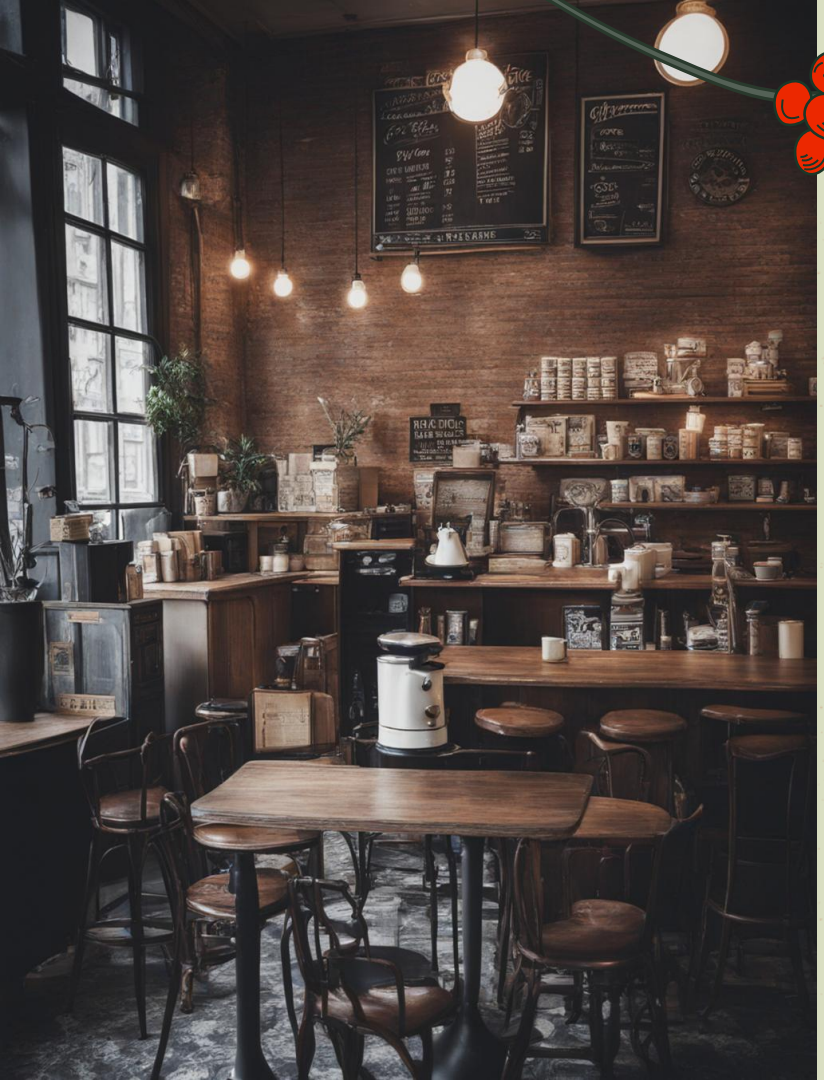
=== Café Inventory Management System ===

What you could expect from this Program:

1. Load inventory data from a file
2. Add new café item
3. Display all café items
4. Display inventory value of café items
5. Display low-stock café items
6. Search café item by name
7. Display café items sorted by price (ascending)
8. Exit program
9. Save inventory data to a file

A Smaple of the Products Information

Product Name	Price	Quantity
CoffeeBeans	45.5	20
GroundCoffee	38.0	15
TeaLeaves	30.0	12
MasalaTea	11.9	13
Nescafe	6.0	15
Latte	6.0	14
HotChocolate	17.0	16
Cocoa	4.5	25
BrownSugar	5.0	18
BreadRolls	6.5	10
Matcha	12.0	8
Croissants	8.0	14
Milk	3.5	22
IceCream	15.0	10



01

Code Blocks of the Project

Project Imports and Requirements

```
#include <iostream>
#include <string>
#include <fstream>

using namespace std;

// Declare and initialize a global variable called `count` to keep track of the products
int count = 0;

// Prototype the functions
void ReadFromFile(string names[], float prices[], int quantities[]);
void AddProduct(string names[], float prices[], int quantities[], int maxSize);
void DisplayItems(string names[], float prices[], int quantities[]);
float InventoryValue(string names[], float prices[], int quantities[], float values[]);
void LowStockProducts(string names[], int quantities[]);
void SearchProduct(string names[], float prices[], int quantities[], float values[]);
void SortProducts(string names[], float prices[], int quantities[], float values[]);
void SaveProducts(string names[], float prices[], int quantities[]);
```

A. Function for Reading from a File

```
void ReadFromFile(string names[], float prices[], int quantities[])
{

    // This function reads data from the text file and stores it in the arrays (parallel arrays).
    ifstream read_file("products.txt");


    while (read_file >> names[count] >> prices[count] >> quantities[count])
    {

        // Read the next product
        count ++;

    }

    read_file.close();

}
```

B. Function for Adding a Product

```
void AddProduct(string names[], float prices[], int quantities[], int maxSize)
{

   // This function gets a product as input and adds its information to the array.
   string name;
   float price;
   int quantity;

   // If the array has reached the maximum size (i.e. 50 items), exit the function
   if (count == maxSize)
   {
      cout << "You reached the maximum size of addition. Choose another option. " << endl;
      return ;
   }

   cout << "Enter the name of the product: ";
   cin >> name;
```

B. Function for Adding a Product (cont.)

```
// Increment the quantity if the product already exists
for (int item = 0; item < count; item++)
{

   if (names[item] == name)
   {

      cout << "The product already exists. Enter the quantity: ";

      cin >> quantity;

      quantities[item] += quantity;

      return ; // Avoid adding the product to the array by exiting the function
   }
}
```

B. Function for Adding a Product (cont.)

```
    // Otherwise, add the product to the array
    cout << "The product does not exist in the array. Enter its price and quantity: ";
    cin >> price >> quantity;


    names[count] = name;
    prices[count] = price;
    quantities[count] = quantity;


    // For each new product increment `count` by one.
    count ++;
    cout << count << endl;
}
```

C. Function for Displaying Items

```
void DisplayItems(string names[], float prices[], int quantities[])
{
   // This function displays all information about the products.
   if (count == 0)
   {
      cout << "No products available. Try adding a few items. “;
   }

   else
   cout << "Product Name | Product Price | Product Quantity “;

   for (int i = 0; i < count; i++)
   {
      cout << names[i] << " | " << prices[i] << " | " << quantities[i];
   }
}
```

D. Function for Calculating Inventory Value

```
float InventoryValue(string names[], float prices[], int quantities[], float values[])
{
    // This function calculates and displays the inventory value of each product.
    string product;
    float value;

    for (int i = 0; i < count; i++)
    {

        value = prices[i] * quantities[i];
        values[i] = value;

    }

    cout << "Enter the name of the product: "; // Display the value of a specific product
    cin >> product;

    for (int item = 0; item < count; item++)
    {
        if (names[item] == product)
        {
            cout << product << " has an inventory value of " << values[item] << endl;
        }
    }

    return value;
}
```

E. Function for displaying Low-Stock Products

```
void LowStockProducts(string names[], int quantities[])
{
    // This function displays the products that have a quantity below a certain threshold (5 units).
    int threshold = 5;
    bool low_stock = false;

    cout << "Products that have a quantity below 5: " << endl;
    for (int item = 0; item < count; item++)
    {
        if (quantities[item] < threshold)
        {
            low_stock = true;
            cout << "Quantity of \'" << names[item] << "\' is " << quantities[item] << endl;
        }
    }

    // No low-stock items
    if (!low_stock)
    cout << "No low-stock products" << endl;
}
```

F. Function for Searching a Product

```
void SearchProduct(string names[], float prices[], int quantities[], float values[])
{
    // This function gets the name of a product from the user and prints the information about that item.
    string product;
    bool isPresent;

    cout << "Enter the product name: ";
    cin >> product;

    // Use Linear Search to find the matching item
    for (int item = 0; item < count; item++)
    {
        isPresent = false;
        if (names[item] == product)
        {
            isPresent = true;
            cout << endl << "Product Name | Product Price | Product Quantity | Product Value: " << endl;
            cout << names[item] << " | " << prices[item] << " | " << quantities[item] << " | " << values[item];
            break;
        }
    }

    // Non-existent product
    if (!isPresent)
    cout << "Product not found" << endl;
}
```

G. Function for Sorting Products

```
void SortProducts(string names[], float prices[], int quantities[], float values[])
{
    // This function uses Bubble Sort to sort the products based on their prices in ascending order.
    bool swapped;

    for (int i = 0; i < (count - 1); i++)
    {
        swapped = false;

        for (int j = 0; j < (count - i - 1); j++)
        {
            if (prices[j] > prices[j+1])
            {
                swap(names[j], names[j+1]);
                swap(prices[j], prices[j+1]);
                swap(quantities[j], quantities[j+1]);
                swap(values[j], values[j+1]);

                swapped = true;
            }
        }
```

G. Function for Sorting Products (cont.)

```
    // If the two elements were not swapped (they were in the correct order), then break from the loop
    if (!swapped)
    {
        break;
    }
  }

  // Display the sorted array
  cout << "Product Name | Product Price | Product Quantity | Product Value: " << endl;

  for (int item = 0; item < count; item++)
  {
    cout << names[item] << " | " << prices[item] << " | " << quantities[item] << " | " << values[item] << endl;
  }
}
```

H. Function for Saving Products

```
void SaveProducts(string names[], float prices[], int quantities[])
{
    // This function saves and stores all the modifications on the products to the text file.
    ofstream write_file("products.txt");

    for (int item = 0; item < count; item++)
    {
        write_file << names[item] << " " << prices[item] << " " << quantities[item] << " " <<  "\n";
    }

    // Close the file
    write_file.close();
}
```

I. Program Flow (Part 1)

```
int main()
{
    // Declare arrays, size, and the user input
    string option;
    const int size = 50;

    string names[size];
    float prices[size];
    int quantities[size];
    float values[size];
    int maxSize = size;

    // Display the Cafe Menu
    cout << R"(=== Cafe Inventory Management System ===
    Welcome to the cafe inventory management system!
    Below you can see what the program can do:
    1. Display the Items
    2. Add a Product
    3. Calculate the Inventory Value of Each Product
    4. Display the Low-Stock Products
    5. Search for a Specific Product
    6. Sort the Products based on their Prices
    7. Exit the Program and Save the Sorted Products in the File
    Feel free to choose an option and play around with the program!
    )";
```

I. Program Flow (Part 2)

```
ReadFromFile(names, prices, quantities);

    // Call the functions based on the user input
    while (true)
    {
        cout << "Enter an option (add, display, value, low-stock, search, sort, exit): ";
        cin >> option;

        if (option == "exit")
        break;
        else if (option == "add")
        AddProduct(names, prices, quantities, maxSize);
        else if (option == "display")
        DisplayItems(names, prices, quantities);
        else if (option == "value")
        InventoryValue(names, prices, quantities, values);
        else if (option == "low-stock")
        LowStockProducts(names, quantities);
        else if (option == "search")
        SearchProduct(names, prices, quantities, values);
        else if (option == "sort")
        SortProducts(names, prices, quantities, values);
        else
        cout << "Invalid option.\n";
    }

    SaveProducts(names, prices, quantities);    // If the user enters 'exit', store the product information and quit the program

    return 0;
}
```

```
=== Cafe Inventory Management System ===

    Welcome to the cafe inventory management system!
    Below you can see what the program can do:

    1. Display the Items
    2. Add a Product
    3. Calculate the Inventory Value of Each Product
    4. Display the Low-Stock Products
    5. Search for a Specific Product
    6. Sort the Products based on their Prices
    7. Exit the Program and Save the Sorted Products in the File

    Feel free to choose an option and play around with the program!

    Enter an option (add, display, value, low-stock, search, sort, exit): display
No products available. Try adding a few items.
Enter an option (add, display, value, low-stock, search, sort, exit):
```

```
Enter an option (add, display, value, low-stock, search, sort, exit): add
Enter the name of the product: Cocoa
The product does not exist in the array. Enter its price and quantity: 4.5 25
Enter an option (add, display, value, low-stock, search, sort, exit): add
Enter the name of the product: Matcha
The product does not exist in the array. Enter its price and quantity: 12.0 8
Enter an option (add, display, value, low-stock, search, sort, exit): add
Enter the name of the product: Milk
The product does not exist in the array. Enter its price and quantity: 3.5 4
Enter an option (add, display, value, low-stock, search, sort, exit): display
Product Name | Product Price | Product Quantity
Cocoa | 4.5 | 25
Matcha | 12 | 8
Milk | 3.5 | 4
Enter an option (add, display, value, low-stock, search, sort, exit): value
Enter the name of the product: Cocoa
Cocoa has an inventory value of 112.5
Enter an option (add, display, value, low-stock, search, sort, exit): low-stock
Products that have a quantity below 5:
Quantity of 'Milk' is 4
```

```
Enter an option (add, display, value, low-stock, search, sort, exit): search
Enter the product name: Matcha

Product Name | Product Price | Product Quantity | Product Value:
Matcha | 12 | 8 | 96
Enter an option (add, display, value, low-stock, search, sort, exit): sort
Product Name | Product Price | Product Quantity | Product Value:
Milk | 3.5 | 4 | 14
Cocoa | 4.5 | 25 | 112.5
Matcha | 12 | 8 | 96
Enter an option (add, display, value, low-stock, search, sort, exit): exit
PS C:\Users\Asus\Downloads\Final Project\ClassicVersion>
```

Resources

- String and character literals (C++)
- Raw String Literal in C++
- Parallel Array
- Return From Void Functions in C++
- swap() in C++
- C++ algorithm swap() function

By Kiana Jafari
CompE Student